

SOP-TSPIPE-001 — Targeted sequencing pipeline (nf-core-tspipe v1.0.0): installation, verification and operation

Document number	SOP-TSPIPE-001
Version	1.0
Applies to software version	nf-core-tspipe v1.0.0 (git tag v1.0.0)
Effective date	
Author	
Reviewed by	
Approved by	
Review interval	Annually, or on any change to section 11
Supersedes	None

1. Purpose

This procedure describes how to install, verify, operate and maintain the nf-core-tspipe targeted sequencing pipeline for clinical myeloid and lymphoid panel analysis, and how to deploy it to an additional server. It exists so that the same input data produces the same clinical result on any qualified installation, and so that any deviation is detectable.

2. Scope

Applies to all installations of nf-core-tspipe v1.0.0 used for clinical reporting, and to all personnel who install, qualify, operate or maintain those installations.

Covers: dependencies and system requirements; installation; installation and operational qualification; routine running; interpretation of run status; troubleshooting; change control; deployment to a new server.

Does not cover: clinical interpretation of variants; wet-lab library preparation or sequencing; the content of the reporting templates.

3. Responsibilities

Role	Responsibility
Bioinformatics operator	Runs the pipeline per section 8; performs pre-run checks; monitors runs; performs first-line troubleshooting per section 10; escalates per section 10.4; retains records per section 13.
Pipeline custodian	Owns the frozen configuration (section 11); authorises and validates all changes; performs installation and qualification (sections 7 and 8) or reviews it; maintains the reference and image manifests.
Reporting pathologist / clinician	Reviews and signs out clinical results. Outside the scope of this SOP.

No person may alter the items listed in section 11 without the custodian's authorisation and a documented re-qualification.

4. Definitions and abbreviations

Term	Meaning
Nextflow	Workflow engine that executes the pipeline and manages task scheduling, caching and resumption.
Process / task	One step of the pipeline for one sample (e.g. alignment of sample X). A full eight-sample run comprises approximately 450 tasks.
Container image	A file containing a complete, fixed software environment (operating system libraries, tools and their dependencies). The pipeline runs every step inside one, so tool versions cannot vary between servers.
Apptainer / Singularity	The container runtime used on HPC systems. Apptainer is the current name; the two are interchangeable here.
Sandbox	A container image unpacked into a directory rather than held as a single .sif file. Required where squashfuse is unavailable (section 6.1).
Executor	The mechanism Nextflow uses to run a task: local (on the machine running Nextflow) or pbspro (submitted to the cluster scheduler).
Profile	A named configuration block selecting the settings for one server (gandalf, clinical23).
Work directory	Nextflow's scratch area, one subdirectory per task, holding the exact command, its log and its outputs. Basis of resumption and of all troubleshooting.
Golden regression	Re-running a fixed reference dataset and comparing the output against an accepted result, to demonstrate that an installation or a change has not altered clinical output.
PoN	Panel of normals — a reference set of normal samples used for copy-number calling.
IQ / OQ	Installation qualification / operational qualification (sections 7.9 and 7.10).

5. System overview

5.1 What the pipeline does

From paired-end FASTQ files it performs adapter trimming and QC, alignment to GRCh38 (masked), duplicate marking, base quality score recalibration and local realignment; calls variants with eight callers and forms a consensus; annotates against VEP, ANNOVAR, VariantValidator, GeneBe, MobiDetails, CancerVar, OncoKB and OncoVI; calls copy number by four independent methods with a consensus, plus B-allele-frequency analysis for copy-neutral loss of heterozygosity; performs dedicated FLT3-ITD detection; and produces per-sample clinical tables, an interactive dashboard, an IGV report and a distributable bundle.

5.2 Design principles relevant to operation

Everything runs in containers. All pipeline steps execute inside container images with recorded checksums. No step depends on software installed on the host beyond those in section 6.1. This is what allows two servers to produce identical output.

The engine and the work are separate. Nextflow (the "head process") runs on one machine and submits the computational work to wherever the site profile directs — a local CPU pool or a cluster scheduler. A small number of steps must run on the head-process machine because they require internet access (section 6.3).

Calling parameters are site-independent. All thresholds that affect which variants are reported live in `conf/modules.config`, which every installation loads. Site profiles contain only paths, executors and resource limits. This separation is mandatory (section 11).

Runs are resumable. Each task is hashed from its inputs, command and container. On resumption, unchanged tasks are reused. Changing a threshold or a container image correctly invalidates the affected tasks.

6. Dependencies and system requirements

6.1 Host software

Component	Required version	Notes
Nextflow	25.10.4	Pinned in <code>nextflow.config</code> ; older versions are rejected.
Java	17 or later	Required by Nextflow.
Apptainer or Singularity	3.6 or later	Must support <code>--env</code> . Apptainer 1.5.1 and Singularity-CE 4.3.2 are qualified.
squashfuse	Optional but recommended	If absent on compute nodes, images must be deployed as sandboxes (section 7.5). Verify on a compute node, not the login node.
Python	3.6 or later	For the installation, verification and comparison utilities only; the pipeline's own Python runs inside the container.
Git, rsync, curl, md5sum	Any current version	Installation and verification.
Docker	Not required	Needed only on the machine that builds the container image (section 7.3).

6.2 Hardware and storage

Resource	Minimum	Qualified configuration
CPU per task	24 cores available to a single task	Compute nodes of 56 cores
Memory per task	96 GB available to a single task	250 GB per node
Head-process machine	8 cores, 32 GB	Login node, 52 cores
Storage for images	50 GB	Sandboxes total approximately 34 GB
Storage for reference data	120 GB	VEP cache 29 GB, ANNOVAR 28 GB, genome and indices 47 GB, hmftools 10 GB
Storage per run	25 GB per sample (work) + 3 GB per sample (outputs)	Shared parallel filesystem

Storage holding reference data, images, work directories and outputs must be visible at the same path on the head-process machine and on every compute node.

6.3 Network

Service	Endpoint	Used by	Required
VariantValidator	https://rest.variantvalidator.org	VARIANT_VALIDATOR	Yes — HGVS nomenclature validation
GeneBe	https://api.genebe.net	DASHBOARD	Yes — ACMG classification
MobiDetails	https://mobidetails.chu-montpellier.fr	DASHBOARD	Yes
CancerVar	https://cancervar.wglab.org	DASHBOARD	Yes
OncoKB	https://www.oncokb.org	DASHBOARD	Yes — requires an API token (section 6.5)

These are accessed only by processes configured to run on the head-process machine. Compute nodes require no internet access.

The head process must therefore run on a machine with outbound HTTPS access. Where a cluster's compute nodes have no name resolution, submitting the head process to the scheduler causes these lookups to fail silently and produce reports with empty annotation sections; this has been observed in practice.

6.4 Container images

Fourteen images are required. They are distributed as files with recorded MD5 checksums; they are not rebuilt at the destination.

Image	Contents
<code>local-tspipe-host-v1.1.img</code>	The principal image: aligners, GATK 4.6.2.0, cnvkit 0.9.12, the variant callers, VEP 105, ANNOVAR scripts, PureCN, DECoN, hmftools, reconCNV, OncoVI, Python and R environments
<code>broadinstitute-gatk-4.5.0.0.img</code>	GATK 4.5.0.0 (Mutect2)
<code>google-deepsomatic-1.10.0.img</code>	DeepSomatic
<code>lethalfang-somaticseq-3.7.4.img</code>	SomaticSeq consensus
<code>local-cava-v2.0.15.img</code>	CAVA annotation
<code>local-filt3r-v0.1.img</code> , <code>local-flt3_itd_ext-v0.2.img</code> , <code>local-getitd-v0.1.img</code>	FLT3-ITD detection
<code>quay.io-biocontainers-*</code> (bcftools, cnvkit, fastp, igv-reports, mosdepth, samtools)	Supporting tools

Authoritative checksums: docs/release/image_checksums.md5 in the repository.

Note that converting a Docker image to Singularity format is not byte-reproducible: two conversions of the same source produce different files. Images must therefore be **copied** from a qualified installation, not regenerated, so that all sites run identical bytes.

6.5 Reference data

Item	Parameter	Approximate size
GRCh38 masked genome, with bwa-mem2 index, .fai, .dict and .alt	reference	47 GB
dbSNP138, Mills indels, gnomAD allele frequencies (with indices)	dbsnp_vcf, mills_vcf, gnomad_af_only	6 GB
VEP cache, release 105, GRCh38	vep_cache	29 GB
ANNOVAR scripts and hg38 tables (refGene, ClinVar, COSMIC, gnomAD exome, avsnip)	annovar_script, annovar_db	28 GB
hmftools resource bundle (AmberGermlineSites, GC profile, Ensembl data, hotspots)	hmf_*	10 GB
Illumina adapter sequences	adapters	small
Panel assets: BED files, PoN, blacklists, CAVA catalogue, reconCNV template	supplied with the repository	small

Authoritative sizes and checksums: docs/release/reference_manifest.tsv.

The .alt file must be present alongside the genome FASTA and named to match it. Its absence changes alignment of ALT contigs and therefore mapping quality.

6.6 Credentials

An OncoKB API token is required for oncogenicity annotation. It is held in ~/.config/nf-core-tspipe/credentials.config, mode 600, in the form:

```
params {
    oncokb_token = '<token>'
}
```

This file is deliberately outside the repository and must never be committed. Without it the pipeline runs to completion but the reports carry no OncoKB annotation.

7. Installation

Perform sections 7.1 to 7.8 in order, then qualify per 7.9 and 7.10. Record each step in the installation record (section 13).

7.1 Verify prerequisites

On the machine that will run the head process:

```
nextflow -version | grep -i version
java -version 2>&1 | head -1
apptainer --version || singularity --version
df -h <storage path>
```

On a compute node, submitted as a job rather than run on the login node — the login node's environment is not representative:

```
hostname; nproc; grep MemTotal /proc/meminfo
command -v apptainer singularity squashfuse unsquashfs
curl -s -m 5 -o /dev/null -w '%{http_code}\n' https://rest.variantvalidator.org/
```

Record: whether the container runtime is on the default PATH in a batch job (if not, note its absolute path for section 7.8); whether squashfuse is present (if not, section 7.5 applies); whether compute nodes have internet access (if not, section 7.8 applies).

7.2 Obtain the pipeline

```
mkdir -p ~/pipelines && cd ~/pipelines
git clone --branch v1.0.0 https://github.com/patkarlab/nf-core-tspipe.git nf-core-tspipe-v1.0.0
cd nf-core-tspipe-v1.0.0 && git log -1 --oneline && git describe --tags
```

Install alongside any existing installation; do not overwrite one. Record the commit hash.

7.3 Obtain the container images

Preferred: copy the image set from a qualified installation (section 7.4), or retrieve it from the archive (section 14.5).

If no qualified installation exists — first ever deployment — the principal image is built on a machine with Docker and the source conda environments, using the scripts in containers/tspipe-host/:

```
bash containers/tspipe-host/pack_envs.sh # package the environments and external installs
bash containers/tspipe-host/build.sh # build, smoke-test, export Docker tar and .img
```

build.sh runs a 29-point smoke test that invokes every tool the pipeline calls and fails the build on any error. It records checksums for the outputs. The remaining thirteen images are pulled from their public registries once and thereafter copied.

7.4 Transfer images and reference data

```
rsync -ah --info=progress2 --partial \
  <source>/singularity_cache/*.img <user>@<host>:<release_root>/images/
rsync -ah --info=progress2 --partial \
  <source>/docs/release/image_checksums.md5 <source>/docs/release/reference_manifest.tsv \
  <user>@<host>:<release_root>/images/
```

Verify on arrival — this is a required check, not optional:

```
cd <release_root>/images && md5sum -c image_checksums.md5 2>&1 | grep -v "No such file"
```

Every image the installation will use must report OK. Any FAILED image must be re-copied. Images present locally from an earlier independent pull will not match and must be replaced.

Transfer reference data the same way for any item not already present at the destination, then verify sizes and checksums against reference_manifest.tsv (section 7.9 automates this).

7.5 Deploy the images in the required form

If `squashfuse` is available on the compute nodes, place the `.img` files in a directory and point `singularity.cacheDir` at it. No conversion is needed.

If `squashfuse` is absent, Apptainer cannot mount a `.sif` and will extract it into local temporary space on every task — which has exhausted a node's storage in practice. Convert each image to a sandbox directory that retains the `.img` name:

```
bash tools/make_sandboxes.sh <release_root>/images <release_root>/sandboxes
```

The script verifies checksums, converts each image, and executes a command inside each sandbox as a functional check. Allow approximately 34 GB and 15 minutes. No pipeline configuration differs between the two forms; only `singularity.cacheDir` changes.

7.6 Place reference data

Reference data may live at any path, provided the path is identical on the head-process machine and all compute nodes. Record the chosen paths for section 7.8.

Confirm the genome's companion files are all present and correctly named:

```
ls <genome>.fai <genome>.dict <genome>.alt <genome>.0123 <genome>.bwt.2bit.64 <genome>.pac <genome>.ann <genome>.amb
```

Where a masked genome is used and only the unmasked `.alt` exists, copy it to the masked name after confirming the checksum matches the qualified installation; the file content is identical.

7.7 Install credentials

```
mkdir -p ~/.config/nf-core-tspipe && chmod 700 ~/.config/nf-core-tspipe
# create credentials.config per section 6.5
chmod 600 ~/.config/nf-core-tspipe/credentials.config
```

7.8 Create the site configuration

Two artefacts are required.

A site profile, `conf/<site>.config`, registered in the `profiles` block of `nextflow.config`. Use `conf/clinical23.config` as the model. It must define, and must define nothing else:

Setting	Guidance
Reference paths	As placed in section 7.6.
<code>process.executor</code>	<code>local</code> for a single server; the scheduler name for a cluster.
<code>process.queue</code>	Scheduler queue names, if applicable.
<code>beforeScript</code>	Only if the container runtime is not on the default <code>PATH</code> in a batch job; export its absolute path.
<code>max_cpus</code> , <code>max_memory</code> , <code>max_time</code>	Must not exceed what a single node can provide, or tasks will never be scheduled.
<code>Executor queueSize</code>	Concurrent tasks. 16 is qualified.
<code>singularity.cacheDir</code>	Directory containing the images or sandboxes.
<code>singularity.runOptions</code>	Bind mounts for every path a task must read or write.
Executor overrides for network-dependent steps	Where compute nodes lack internet, <code>DASHBOARD</code> , <code>REPORT_BUNDLE</code> , <code>RUN_BUNDLE</code> and <code>ORGANIZE_OUTPUT</code> must be given <code>executor = 'local'</code> with resource requests that fit the local pool. <code>VARIANT_VALIDATOR</code> is already pinned to <code>local</code> in <code>conf/modules.config</code> .
<code>includeConfig 'containers.config'</code>	Last line. Assigns container images and per-process environment paths.

Do not place calling thresholds, filtering parameters or tool arguments in a site profile (section 11).

Two failure modes observed during commissioning, both silent:

- A scheduler directive supplied through `clusterOptions` that contains a resource request causes Nextflow to omit its own resource request, and tasks then run with the scheduler's defaults and are killed. Do not put resource requests in `clusterOptions`.
- A process pinned to the `local` executor but requesting more CPUs than the local pool provides aborts the run. Size the local pool to the largest such request, or give those processes explicit smaller requests.

A site parameter file, `params_<site>.yaml`, passed with `-params-file`. At minimum it carries the `hmftools` reference paths and the `VariantValidator` cache directory.

Configuration precedence decides what this file is for, and it is not obvious:

Source	Precedence
<code>--parameter</code> on the command line	highest
<code>-params-file</code>	overrides everything below

-c <overlay>.config (a panel overlay)	overrides the profile
The site profile selected by -profile	overrides the defaults
nextflow.config and conf/modules.config	defaults

A panel overlay is loaded **after** the site profile, so any parameter the overlay sets cannot be changed from a site profile — whatever the profile says is discarded. The hmftools reference paths in `conf/twist_apply.config` are of this kind: they are hardcoded to the development host, and a second site redirects them with its parameter file, not with its profile. A conditional guard in the overlay was tried and does not work: the guard evaluates before the profile's value is visible, so the overlay's default always wins (verified on clinical-23, 2026-09-11).

The practical consequence is that a run without `-params-file` uses the development host's hmftools paths and fails at AMBER. The site launcher (section 8.3) always passes it, which is why runs are started through the launcher rather than by hand.

7.9 Installation qualification

```
bash tools/verify_install.sh -profile <site>,singularity \
-c conf/<panel>.config -params-file params_<site>.yaml
```

Checks the toolchain and versions; every image against `image_checksums.md5`; every reference against `reference_manifest.tsv` including checksums; the genome's companion files; and that the output locations are writable.

Acceptance criterion: the check reports VERIFY PASSED. Warnings for images this configuration does not use, and for default paths belonging to another site, are acceptable. Retain the output.

Then confirm the workflow itself is coherent without consuming compute:

```
nextflow run . -stub -profile <site>,singularity -c conf/<panel>.config \
-params-file params_<site>.yaml --input <samplesheet> --outdir <tmp> -w <tmp_work>
```

Acceptance criterion: the stub run completes without error.

7.10 Operational qualification — golden regression

Run the eight-sample validation cohort and compare against the accepted result from a qualified installation.

```
setuid nextflow run . --input <validation samplesheet> \
--outdir <path>/oq_<date> -w <path>/work_oq_<date> \
-profile <site>,singularity -c conf/<panel>.config \
-params-file params_<site>.yaml -ansi-log false \
> /tmp/oq_<date>.log 2>&1 < /dev/null & disown

python3 tools/compare_runs.py --a <accepted reference outdir> --b <path>/oq_<date> \
--label-a accepted --label-b new --out <path>/oq_<date>_comparison.md
```

Acceptance criteria:

1. The run completes with no failed tasks, ending with `Cleanup complete`.
2. For every sample, these are byte-identical to the accepted reference: `- clinical/<sample>.somaticseq.clinical.final.tsv - clinical/<sample>.somaticseq.filtered.tsv - clinical/cnv/consensus/*.genes.tsv - clinical/cnv/baf/*.summary.tsv` - the PURPLE, CNVkit, DECoN and GATK CNV tables
3. Differences are confined to: log files containing timestamps (AMBER, COBALT, PURPLE, HsMetrics, quickcheck); files that embed a creation time (ZIP archives, PDF, `.rds`, HTML reports); and annotation caches holding responses from external services.

Any difference in a clinical table fails the qualification. Do not release the installation for clinical use; escalate to the custodian with the comparison report.

Retain the comparison report as the qualification record.

8. Routine operation

8.1 Prepare the samplesheet

CSV with header, one line per sample:

```
sample,fastq_1,fastq_2,sex
<sample id>,<absolute path R1>,<absolute path R2>,<male|female|unknown>
```

- `sample` becomes the output directory name and appears in every report; use the laboratory identifier exactly.
- FASTQ paths must be absolute and readable from the compute nodes.
- `sex` selects the sex-matched panel of normals for copy-number calling. Supply it when known; `unknown` is permitted and the pipeline reports inferred sex.

Verify every file exists before launching:

```
awk -F, 'NR>1{print $2"\n"$3}' <samplesheet> | while read f; do [ -f "$f" ] || echo "MISSING $f"; done
```

8.2 Pre-run checks

1. Free space: `df -h <storage>` — allow 25 GB per sample for the work directory and 3 GB per sample for outputs.
2. The installation is unmodified: `git -C <pipeline dir> status --short` shows no modified tracked files (untracked samplesheets and the site parameter file are expected).
3. No other run is using the intended work directory.

8.3 Launch

Use the site launcher. Each installation ships one — `tools/launch_<site>.sh` — and it is the supported way to start a run:

```
cd <pipeline dir>
bash tools/launch_<site>.sh <samplesheet.csv> <run name> [extra nextflow args]
```

It supplies the site profile, the panel overlay, the parameter file and the VariantValidator endpoint; validates the samplesheet and every FASTQ path; checks free space and that no run is already active against the same work directory; runs the endpoint preflight; and refuses to start from a batch job or from any machine other than the head-process machine. It then launches detached and confirms the run started. Extra arguments, such as `-resume`, are passed through.

`launch_tspipe.sh` in the repository root is the **development host's** launcher: its defaults select the development profile and a run started with it on another site fails with missing references under that host's paths. This is not a fault in the installation; it is the wrong launcher. Do not edit it to suit a site — add or use that site's launcher instead.

The underlying command, for reference and for troubleshooting only:

```
RUN=<run name>
setuid nextflow run . \
  --input ${RUN}.csv \
  --outdir <path>/${RUN} \
  -w <path>/work_${RUN} \
  -profile <site>,singularity \
  -c conf/<panel>.config \
  -params-file params_<site>.yaml \
  -ansi-log false \
  > /tmp/${RUN}.log 2>&1 < /dev/null & disown
```

- `setuid ... & disown` detaches the process so it survives logout. Always use it.
- `-ansi-log false` produces a readable log file.
- Do not submit this command to the scheduler where compute nodes lack internet access (section 6.3).
- One run per work directory. A second launch against the same work directory fails with `Unable to acquire lock on session`.

Confirm dispatch after one minute:

```
grep -E "Submitted process|ERROR" /tmp/${RUN}.log | tail -3
```

8.4 Monitor

```
grep -cE "Submitted process" /tmp/${RUN}.log          # tasks dispatched
grep -E "ERROR|terminated with" /tmp/${RUN}.log | tail # failures
qstat -u <user> | grep -c nf-TSPIPE                  # scheduler jobs (cluster installations)
pgrep -u <user> -f "${RUN}" | wc -l                   # head process alive
```

An eight-sample run comprises approximately 450 tasks and takes two to three hours, depending on competing load. Two stages appear stalled and are not: `samtools mpileup` within VARSCAN takes 20–30 minutes per sample, and VARIANT_VALIDATOR is deliberately serialised against a rate-limited public service.

8.5 Completion

The run is complete when the log contains:

```
Cleanup complete. Final per-sample layout: <outdir>/<sample>/clinical/
```

and no scheduler jobs or head process remain. A run that ends without this line did not finish; treat it per section 10.

8.6 Outputs and review

Per sample, under `<outdir>/<sample>/`:

Path	Content
<code>clinical/<sample>.somaticseq.clinical.final.tsv</code>	Final annotated clinical variant table
<code>clinical/<sample>.somaticseq.filtered.tsv</code>	Full filtered variant table
<code>clinical/<sample>_report.html, _dashboard.html</code>	Reports for review
<code>clinical/<sample>_igv_report.html</code>	Read-level visualisation of reported variants
<code>clinical/<sample>.final.bam</code>	Analysis-ready alignment
<code>clinical/cnv/</code>	Copy-number consensus, BAF, PURPLE, CNVkit, DECoN, reconCNV
<code><sample>_report.zip</code>	Self-contained bundle for distribution
<code>qc/, sex_check/</code>	Quality control outputs

Cohort level: `cohort_index.html` and `<run>_reports.zip`.

Before release, confirm: the completion line is present; every expected sample has a `clinical/` directory; QC metrics are within laboratory acceptance criteria; and the dashboard shows populated annotation sections (empty GeneBe, MobiDetails or OncoKB sections indicate a network or credential fault, not an absence of findings).

8.7 Retention and cleanup

Retain the output directory and the run log per section 13. Once outputs are verified and archived, the work directory may be deleted:

```
rm -rf <path>/work_<run>
```

Outputs are independent files, not links, and are unaffected. Deleting the work directory prevents any later resumption of that run.

9. Interrupting and resuming

To stop: `pkill -u <user> -f "<run name>"`. Nextflow completes tasks already dispatched before exiting, which may take several minutes; allow it to finish so that the cache is written.

To resume: re-issue the launch command with `-resume` and the same work directory. Unchanged tasks are reused and reported as `Cached process`; only failed or invalidated tasks re-run.

Resumption is valid only if the work directory is intact and the pipeline, images and parameters are unchanged. A deliberate parameter change correctly invalidates the affected tasks — this is the mechanism by which a corrected threshold propagates.

10. Troubleshooting

10.1 Locating the cause

Nextflow names the failing process and its work directory:

```
grep -A25 "Error executing process" /tmp/<run>.log | head -35
```

The task directory contains `.command.sh` (the exact command), `.command.log` (its output), `.command.run` (the wrapper, including scheduler directives and the container invocation) and `.exitcode`. These four files are sufficient to diagnose almost any failure.

10.2 Known failure modes

Symptom	Cause	Action
terminated with an error exit status (137)	Task killed, almost always for exceeding its memory allocation	Inspect <code>#PBS -l select=</code> in <code>.command.run</code> . If it names <code>no mem</code> , a resource request was suppressed by a configuration fault — escalate. Otherwise the task genuinely exceeded its limit — escalate for a resource review.
Process requirement exceeds available CPUs — req: N; avail: M	A process on the local executor requests more CPUs than the local pool provides	Escalate. Indicates a process has moved between executors or the pool is undersized.
Unable to acquire lock on session	A previous Nextflow is still exiting against the same work directory	Wait until no Nextflow process remains, then resume.
Unknown configuration profile	The site profile is not registered in <code>nextflow.config</code>	Escalate.
VariantValidator task fails	External service unavailable or rate-limiting	Three automatic retries are configured. If it still fails, test with <code>curl -s -m 10 -o /dev/null -w '%{http_code}\n' https://rest.variantvalidator.org/</code> ; if the service is down, resume later. Cached responses are not re-requested.
Reports contain empty GeneBe / MobiDetails / CancerVar sections	The dashboard executed without internet access	Escalate — the head process or the dashboard step is running in the wrong place. Results are incomplete and must not be released.
Reports contain no OncoKB annotation	Credentials absent or invalid	Verify <code>~/config/nf-core-tspipe/credentials.config</code> per section 6.6. Re-run the dashboard step.
A task fails once and succeeds on resume	Transient infrastructure fault	Acceptable. Record it. If the same task fails twice at the same point, escalate.
Run stops with no error and no completion line	Head process terminated (logout without <code>setuid</code> , node restart, out of disk)	Check disk, then resume.
Tasks queue but never start	Requested resources exceed any node, or the queue is full	Compare the profile's <code>max_cpus</code> / <code>max_memory</code> against node capacity; check scheduler queue status.

10.3 Routine diagnostic checks

```
bash tools/verify_install.sh -profile <site>,singularity -c conf/<panel>.config -params-file params_<site>.yaml
git -C <pipeline dir> status --short
ls <release_root>/sandboxes/ | wc -l
df -h <storage>
```

10.4 Escalation

Escalate to the pipeline custodian with: run name; the run log; the failing process name and its work directory path; and the output of section 10.3.

Do not modify the pipeline, container images, reference data or parameters to work around a failure. A result produced by an undocumented change cannot be released clinically and invalidates the installation's qualification.

11. Frozen configuration and change control

The following were validated together and constitute the qualified state:

1. The pipeline at tag `v1.0.0`, unmodified.
2. The fourteen container images, matching `docs/release/image_checksums.md5`. These cannot be reproduced; they are backed up per section 14.
3. All calling and filtering thresholds in `conf/modules.config` — notably VarScan's `min_var_freq` of 0.03. During commissioning this

parameter was found in a site profile rather than the shared configuration, and a second installation consequently called at a tenfold lower threshold and reported additional variants. Thresholds must never reside in a site profile.

4. The reference data, matching `docs/release/reference_manifest.tsv`.
5. Nextflow 25.10.4.
6. The site launcher and the site parameter file. A run started by any other means may silently use another site's configuration.

Any change to these requires: authorisation by the custodian; implementation on the development installation first; a golden regression per section 7.10 with the comparison report retained; a new version tag; and an update to this SOP's revision history. Changes to site profiles — paths, executors, resource limits — do not require re-validation of calling behaviour but do require the checks in sections 7.9 and 7.10 to be repeated.

12. Deploying to an additional server

Follow section 7 in full. The following decisions are site-specific and must be recorded in the installation record:

Decision	Determined by
Where the head process runs	Which machine has outbound HTTPS access (section 6.3)
Image form: <code>.sif</code> or <code>sandbox</code>	Presence of <code>squashfuse</code> on compute nodes (section 7.5)
Executor and queues	Local server or cluster scheduler
Resource ceilings	Largest single node
Container runtime invocation	Whether the runtime is on the default <code>PATH</code> in a batch job
Bind mounts	Every path a task reads or writes
Reference data locations	Storage layout; must be identical across all nodes

Everything else — pipeline, images, thresholds, reference content — is copied unchanged. A deployment is complete only when section 7.10 has passed and the comparison report is retained.

13. Records

Record	Retained by	Retention
Installation record: date, operator, commit hash, image checksums, reference locations, site decisions per section 12	Custodian	Life of the installation
Installation qualification output (section 7.9)	Custodian	Life of the installation
Operational qualification comparison report (section 7.10)	Custodian	Life of the installation, and after every change
Run log (<code>/tmp/<run>.log</code> , copied to the output directory)	Operator	Per laboratory policy for clinical records
Run outputs	Operator	Per laboratory policy for clinical records
Backup verification and restore-test records (section 14)	Custodian	Life of the installation
Deviation and escalation records	Custodian	Per laboratory policy

14. Backup, archive and restore of the container images

14.1 Why this matters

The container images are the only components of the system that cannot be reproduced. Converting a Docker image to Singularity format is not byte-reproducible: rebuilding from the same source yields a different file, and an installation running different bytes is no longer the qualified installation. The reference data can be re-downloaded and the pipeline is in version control; the images cannot be recovered from anywhere except a backup.

Three copies are maintained, on independent systems, each verified by checksum:

Copy	Location	Purpose
Primary	<code>/goast/hemat_data/tspipe_release/backup_v1.0.0</code> on the development host	Working copy; source for new deployments
Secondary	<code>/scratch/patkarlab-clinical/tspipe_release/backup_v1.0.0</code> on <code>clinical-23</code>	Survives loss of the development host
Archive	<code>s3://hemat/tspipe_release/v1.0.0/</code> (ACTREC Dell ECS, endpoint <code>caib-ecs.actrec.gov.in</code>)	Survives loss of either compute system

The archive bucket is institutional object storage, not a public service. The images bundle ANNOVAR, VEP, Strelka2, OncoVI and the `hmftools` jars, whose licences do not permit public redistribution: they must never be placed on a public registry or any world-readable location.

14.2 What the backup contains

Path within the backup	Content
<code>images/</code>	All 14 Singularity images (<code>.img</code>), plus the Docker export of the principal image (<code>local-tspipe-host-v1.1.docker.tar.gz</code>) and its checksum
<code>src/</code>	The <code>conda-pack</code> tarballs from which the principal image was built, with <code>CHECKSUMS.md5</code> . These are excluded from version control and exist nowhere else
<code>docs/</code>	Release notes, image checksums, reference manifest, this SOP, and <code>COMMIT.txt</code> recording the exact commit and tag
<code>nf-core-tspipe-v1.0.0.gitbundle</code>	A complete clone of the repository, so the pipeline is recoverable without access to the hosting service
<code>MANIFEST.md5</code>	Checksums of every file above

Total approximately 27 GB.

14.3 Creating or refreshing the backup

Performed on the development host by the custodian, after any change to the image set:

```
cd <pipeline dir>
B=<release root>/backup_<version>
mkdir -p $B/images $B/src $B/docs
cp -p <singularity cache>/*.img $B/images/
cp -p <release>/images/*.docker.tar.gz* $B/images/
cp -p containers/tspipe-host/src/*.tar.gz containers/tspipe-host/src/CHECKSUMS.md5 $B/src/
cp -p docs/release/image_checksums.md5 docs/release/reference_manifest.tsv docs/RELEASE_NOTES.md docs/sops/SOP-TSPIPE-001.pdf
docs/sops/SOP-TSPIPE-001.md $B/docs/
git bundle create $B/nf-core-tspipe-<version>.gitbundle --all
git log -l --oneline > $B/docs/COMMIT.txt && git describe --tags >> $B/docs/COMMIT.txt
cd $B && find . -type f ! -name MANIFEST.md5 -exec md5sum {} \; | sort -k2 > MANIFEST.md5
md5sum -c MANIFEST.md5 | grep -c "": OK"
```

Distribute to the secondary and archive copies, verifying each:

```
rsync -ah --info=progress2 $B/ <user>@<clinical host>:<path>/backup_<version>/
ssh <user>@<clinical host> 'cd <path>/backup_<version> && md5sum -c MANIFEST.md5 | grep -c "": OK"'

s3cmd put --recursive $B/ s3://hemat/tspipe_release/<version>/
s3cmd ls --recursive s3://hemat/tspipe_release/<version>/ | wc -l
```

Both verification commands must report the file count in MANIFEST.md5.

14.4 Verifying the archive copy

Checksum an object by downloading it to a file. Do not attempt to verify by streaming to standard output — `s3cmd get ... -` does not deliver the object content and produces a misleading checksum.

```
s3cmd get --force s3://hemat/tspipe_release/v1.0.0/images/local-tspipe-host-v1.1.img /tmp/verify.img
ls -la /tmp/verify.img; md5sum /tmp/verify.img; rm -f /tmp/verify.img
```

Expected: 3,534,614,528 bytes, MD5 `aede69a3185ea55ffe15c813ee97af07`.

Verify the archive at least annually and after any change to the image set, and record the result.

14.5 Retrieving the images for a new or rebuilt installation

This replaces section 7.3 when no qualified installation is reachable to copy from.

```
mkdir -p <release root>/images && cd <release root>/images
s3cmd get --recursive s3://hemat/tspipe_release/v1.0.0/images/ .
s3cmd get s3://hemat/tspipe_release/v1.0.0/MANIFEST.md5 .
s3cmd get --recursive s3://hemat/tspipe_release/v1.0.0/docs/ ../docs/
md5sum -c ../MANIFEST.md5 2>/dev/null | grep -vE "": OK$" | head
```

Every image must report `OK`. Then proceed from section 7.5 (deploy as `.sif` or sandboxes), section 7.6 (reference data), section 7.7 (credentials) and section 7.8 (site configuration).

If the repository is also unavailable, restore it from the bundle:

```
s3cmd get s3://hemat/tspipe_release/v1.0.0/nf-core-tspipe-v1.0.0.gitbundle /tmp/
git clone /tmp/nf-core-tspipe-v1.0.0.gitbundle <pipeline dir>
cd <pipeline dir> && git checkout v1.0.0 && git describe --tags
```

14.6 Installing from the Docker export

Required only where the destination uses Docker rather than Apptainer, or where a Singularity image must be regenerated locally. Note that a locally regenerated `.sif` will not be byte-identical to the archived one; the installation must then be qualified by section 7.10 rather than by checksum alone.

```
s3cmd get s3://hemat/tspipe_release/v1.0.0/images/local-tspipe-host-v1.1.docker.tar.gz /tmp/
s3cmd get s3://hemat/tspipe_release/v1.0.0/images/local-tspipe-host-v1.1.docker.tar.gz.md5 /tmp/
cd /tmp && md5sum -c local-tspipe-host-v1.1.docker.tar.gz.md5
docker load -i local-tspipe-host-v1.1.docker.tar.gz
docker images local/tspipe-host
docker run --rm -u "$(id -u):$(id -g)" \
  -v <pipeline dir>/containers/tspipe-host/smoke_test.sh:/tmp/smoke_test.sh:ro \
  local/tspipe-host:v1.1 bash /tmp/smoke_test.sh
```

The smoke test must report 29 successful checks. To produce a Singularity image from it:

```
singularity build <cache>/local-tspipe-host-v1.1.img docker-daemon://local/tspipe-host:v1.1
```

14.7 Restore testing

A backup that has never been restored is an assumption. At least once per release, and after any change to the image set, perform 14.6 on a machine that does not hold the original, run the smoke test, and record the outcome with the qualification records (section 13).

15. Revision history

Version	Date	Author	Change
1.0			First issue. Covers nf-core-tspipe v1.0.0 as validated on two installations (2026-09-10 and 2026-09-11), including the image backup and archive procedure (section 14).
1.1			Covers v1.0.1. Adds the configuration precedence rule and why a site parameter file, not a site profile, redirects a parameter set by a panel overlay (7.8); makes the site launcher the supported way to start a run, with the failure produced by using another site's launcher (8.3, A.3); adds the launcher and parameter file to the frozen configuration (11).

16. Associated documents

Document	Location
Release notes, including known findings and deferred items	docs/RELEASE_NOTES.md
Container image checksums	docs/release/image_checksums.md5
Reference data manifest	docs/release/reference_manifest.tsv
Commissioning and validation records	docs/audit/2026-09-10/, docs/audit/2026-09-11/
Pipeline output specification	docs/output.md
Panel and dashboard procedures	docs/sops/
clinical-23 installation and operating instructions	Annex A of this document
Image backup and archive	s3://hemat/tspipe_release/v1.0.0/; see section 14
Clinical parameter decisions	docs/clinical_decisions.md

Annex A — clinical-23: installation as built, and operating instructions

This annex records the clinical-23 installation as it was actually performed and qualified, and gives the exact procedure for running analyses on it. Nothing in A.3 onwards requires installation work: the system described in A.1 is in place and qualified.

A.1 Installation record

Item	Value
Installed and qualified	2026-09-11
Pipeline	~/pipelines/nf-core-tspipe-v1.0.0, git tag v1.0.0
Account	patkarlab-clinical
Head process runs on	ln1 (login node — the only machine with outbound HTTPS)
Compute	PBS Pro; queue medium, and short for light tasks; 46 nodes of 56 cores / 250 GB
Resource ceilings	48 CPUs, 200 GB, 72 h per task; 16 concurrent tasks
Container runtime	Apptainer 1.5.1 at /soft/apptainer/1.5.1/bin/apptainer, placed on PATH by the profile's beforeScript (the module system does not load reliably in batch jobs)
Image form	Sandbox directories — compute nodes have no squashfuse
Images	/scratch/patkarlab-clinical/tspipe_release/images/ (14 .img, all verified against docs/release/image_checksums.md5)
Sandboxes	/scratch/patkarlab-clinical/tspipe_release/sandboxes/ (14 directories, approx. 34 GB)
Genome, VCFs, VEP cache	~/references/hg38_broad/, ~/references/dbSNPGATK_hg38/, ~/references/vep_cache/ (VEP release 105, GRCh38)
ANNOVAR	~/programs/annovar/ (scripts and hg38 tables: refGene, ClinVar 20220320, COSMIC 103, gnomAD exome 211, avsn150)
hmftools bundle	/scratch/patkarlab-clinical/tspipe_release/refs/hmftools/hmf_pipeline_resources.38_v3.0.0-8/
Adapters	~/references/adapters/illumina_adapters.fa
Site profile	conf/clinical23.config
Site parameters	~/pipelines/nf-core-tspipe-v1.0.0/params_clinical23.yaml
Site launcher	tools/launch_clinical23.sh — the supported way to start a run here
VariantValidator cache	/scratch/patkarlab-clinical/tspipe_release/vv_cache/
Credentials	~/config/nf-core-tspipe/credentials.config (OncoKB token, mode 600)
Image backup (secondary copy)	/scratch/patkarlab-clinical/tspipe_release/backup_v1.0.0 (27 GB, verified 2026-09-11)
Image backup (archive copy)	s3://hemat/tspipe_release/v1.0.0/
Accepted reference run	/scratch/patkarlab-clinical/tspipe_run8_c23 (8 Twist validation samples)
Qualification	IQ and OQ passed 2026-09-11. Clinical tables byte-identical to the gandalf installation; comparison report docs/audit/2026-09-11/run8_gandalf_vs_clinical23_v2.md

A.2 How the installation was performed

Recorded for audit and for anyone repeating it. All steps were executed as patkarlab-clinical.

- Pipeline.** Cloned at the release tag into a new directory, leaving the previous installation (~/pipelines/nf-core-tspipe) untouched:
git clone --branch v1.0.0 <repo> ~/pipelines/nf-core-tspipe-v1.0.0
- Images.** Copied from the qualified gandalf installation by rsync over SSH, together with image_checksums.md5 and reference_manifest.tsv. Nine images already present on ln1 from earlier independent pulls did **not** match — Singularity conversion is not byte-reproducible — and were replaced by the gandalf copies. All fourteen then verified OK.

3. **Image form.** A test job on a compute node established that Apptainer is absent from the default `PATH` in a batch shell but present at `/soft/apptainer/1.5.1/bin/apptainer`, and that `squashfuse` is not installed. Images were therefore converted to sandboxes with `tools/make_sandboxes.sh`, each retaining its `.img` name; all fourteen passed the post-conversion execution check.
4. **Reference data.** The genome, its `bwa-mem2` index, the Broad VCFs, the VEP 105 cache and the ANNOVAR tables were already present on `ln1` and matched the manifest. Transferred from `gandalf`: the `hmftools` resource bundle (9.8 GB) and `illumina_adapters.fa`. The masked genome had no `.alt` file; the Broad `.alt` was copied to the masked name after confirming its checksum (`b07e65aa4425bc365141756f5c98328c`) matched `gandalf`'s.
5. **Site configuration.** `conf/clinical23.config` written and registered in the `profiles` block of `nextflow.config`. `params_clinical23.yaml` created to redirect the `hmftools` reference paths, which are hardcoded to `gandalf` locations in the Twist panel overlay, and to set the `VariantValidator` cache directory. The `VariantValidator` cache directory was created.
6. **Credentials.** The OncoKB token was installed at `~/.config/nf-core-tspipe/credentials.config` with mode 600. The previous installation had held a placeholder value, so OncoKB annotation had never been produced on this server before.
7. **Installation qualification.** `tools/verify_install.sh` run against the profile and parameter file; reported `VERIFY PASSED`.
8. **Operational qualification.** The eight-sample Twist validation cohort was run and compared against the `gandalf` result on the same image. The first comparison exposed a genuine defect — `VarScan`'s `min_var_freq` was held in the `gandalf` site profile rather than the shared configuration, so this installation called at the module default of 0.003 and reported additional variants. The parameter was moved to `conf/modules.config`, both installations were re-run, and the clinical tables then matched byte for byte.

`params_clinical23.yaml` contains:

```
hmf_resources: "/scratch/patkarlab-clinical/tspipe_release/refs/hmftools/hmf_pipeline_resources.38_v3.0.0--8"
hmf_loci: "<hmf_resources>/dna/copy_number/AmberGermlineSites.38.tsv.gz"
hmf_gc_profile: "<hmf_resources>/dna/copy_number/GC_profile.1000bp.38.cnp"
hmf_ensembl_dir: "<hmf_resources>/common/ensembl_data"
hmf_hotspots: "<hmf_resources>/dna/variants/KnownHotspots.somatic.38.vcf.gz"
vv_cache_dir: "/scratch/patkarlab-clinical/tspipe_release/vv_cache"
```

(paths written in full in the file; abbreviated here for readability).

A.3 Running a Twist myeloid panel analysis

Step 1 — log in to `ln1` as `patkarlab-clinical`. Runs are launched from the login node, not submitted to the queue. The head process must stay on `ln1` because the dashboard's GeneBe, MobiDetails, CancerVar and OncoKB lookups and the `VariantValidator` step need internet access, which compute nodes do not have. Everything computational is submitted to PBS automatically.

Step 2 — place the FASTQ files on `/scratch`, for example `/scratch/patkarlab-clinical/<batch>/`. Do not run from the home directory.

Step 3 — write the samplesheet in the pipeline directory, one line per sample:

```
cd ~/pipelines/nf-core-tspipe-v1.0.0
cat > myrun.csv <<'EOF'
sample,fastq_1,fastq_2,sex
26ABC123-Twist,/scratch/patkarlab-clinical/batch12/26ABC123_R1.fastq.gz,/scratch/patkarlab-clinical/batch12/26ABC123_R2.fastq.gz,male
26ABC124-Twist,/scratch/patkarlab-clinical/batch12/26ABC124_R1.fastq.gz,/scratch/patkarlab-clinical/batch12/26ABC124_R2.fastq.gz,female
EOF
awk -F, 'NR>1{print $2"\n"$3}' myrun.csv | while read f; do [ -f "$f" ] || echo "MISSING $f"; done
```

Nothing printed by the last line means every file is present.

Step 4 — pre-run checks:

```
df -h /scratch # 25 GB per sample for work, 3 GB for outputs
git -C ~/pipelines/nf-core-tspipe-v1.0.0 status --short # only untracked samplesheets expected
pgrep -u patkarlab-clinical -f nextflow | wc -l # 0 unless another run is intended
```

Step 5 — launch:

```
cd ~/pipelines/nf-core-tspipe-v1.0.0
bash tools/launch_clinical23.sh myrun.csv myrun_20260915
```

The launcher supplies the profile, the Twist panel overlay, the site parameter file and the `VariantValidator` endpoint; checks the samplesheet and every FASTQ path; verifies free space and that no run is already active; runs the endpoint preflight; and refuses to start from a batch job or from any machine other than `ln1`. It launches detached and confirms the run started. Add `-resume` as a third argument to continue an interrupted run.

Do not start runs with `launch_tspipe.sh`. That is the development host's launcher: it defaults to the `gandalf` profile, so every reference resolves under `/goast`, which does not exist here, and the run fails immediately with missing files. This occurred on 2026-09-11 and was initially mistaken for a reference problem. The installation was correct; the launcher was the wrong one.

The command the launcher issues, for reference when troubleshooting:

```
setsid nextflow run . --input myrun.csv \
--outdir /scratch/patkarlab-clinical/myrun_20260915 \
-w /scratch/patkarlab-clinical/work_myrun_20260915 \
-profile clinical23,singularity -c conf/twist_apply.config \
-params-file params_clinical23.yaml -ansi-log false \
> /tmp/myrun_20260915.log 2>&1 < /dev/null & disown
```

`-params-file params_clinical23.yaml` is not optional: it redirects the `hmftools` reference paths, which the panel overlay hardcodes to the development host (section 7.8), and a run without it fails at AMBER.

Step 6 — confirm it started, after about a minute:

```
grep -E "Submitted process|ERROR" /tmp/${RUN}.log | tail -3
qstat -u patkarlab-clinical | tail -3
```

A.4 Monitoring and completion

```
grep -cE "Submitted process" /tmp/${RUN}.log          # tasks dispatched
grep -E "ERROR|terminated with" /tmp/${RUN}.log | tail # failures
qstat -u patkarlab-clinical | grep -c nf-TSPIPE       # jobs on the cluster
pgrep -u patkarlab-clinical -f "${RUN}" | wc -l       # head process alive
```

An eight-sample run is approximately 450 tasks and takes two to three hours. The run is complete when the log contains `Cleanup complete`. Final per-sample layout: `<outdir>/<sample>/clinical/` and no `nf-TSPIPE` jobs remain.

Site-specific behaviour worth knowing:

- **Other pipelines share this cluster.** Jobs named other than `nf-TSPIPE` in the queue belong to other analyses; competing load lengthens a run without harming it.
- **PBS packs several tasks onto one node.** Eight tasks appearing on a single node is normal.
- **Two stages look stalled and are not.** `samtools mpileup` inside `VARSCAN` takes 20–30 minutes per sample and writes a temporary file of one to two gigabytes; `VARIANT_VALIDATOR` is deliberately serialised against a rate-limited public service.
- **The head process holds a session lock.** After stopping a run, wait until no Nextflow process remains before resuming, or the resume fails with `Unable to acquire lock on session`.

A.5 Outputs

Under `/scratch/patkarlab-clinical/<RUN>/`:

Path	Content
<code><sample>/clinical/<sample>.somaticseq.clinical.final.tsv</code>	Final annotated clinical variant table
<code><sample>/clinical/<sample>_report.html, _dashboard.html, _igv_report.html</code>	Reports for review
<code><sample>/clinical/cnv/</code>	CNV consensus, BAF, PURPLE, CNVkit, DECoN, reconCNV
<code><sample>/clinical/<sample>.final.bam</code>	Analysis-ready alignment
<code><sample>/<sample>_report.zip</code>	Self-contained per-sample bundle
<code>cohort_index.html, <RUN>_reports.zip</code>	Cohort index and combined bundle

Before release, confirm the completion line is present, every sample has a `clinical/` directory, QC is within laboratory criteria, and the dashboard's annotation sections are populated — empty `GeneBe`, `MobiDetails` or `OncoKB` sections indicate a network or credentials fault, not an absence of findings.

A.6 Stopping, resuming, cleaning up

```
pkill -u patkarlab-clinical -f "${RUN}"                # stop; allow it to finish exiting
while pgrep -u patkarlab-clinical -f "${RUN}" >/dev/null; do sleep 20; done; echo exited
```

Resume with the identical launch command plus `-resume`. Completed tasks are reused and reported as `Cached process`.

After outputs are verified and archived:

```
rm -rf /scratch/patkarlab-clinical/work_${RUN}
```

Outputs are independent files and are unaffected; the run can no longer be resumed afterwards.

A.7 Re-qualification on clinical-23

After any change to the installation, repeat the operational qualification with the validation cohort already present on this server:

```
cd ~/pipelines/nf-core-tspipe-v1.0.0
RUN=twistval_$(date +%Y%m%d)
bash tools/launch_clinical23.sh twist_val_8_clinical23.csv ${RUN}
```

When it completes, compare against the accepted reference run:

```
python3 tools/compare_runs.py \
  --a /scratch/patkarlab-clinical/tspipe_run8_c23 \
  --b /scratch/patkarlab-clinical/${RUN} \
  --label-a accepted --label-b new \
  --out /scratch/patkarlab-clinical/${RUN}_vs_accepted.md
```

Acceptance criteria are those of section 7.10. Retain the comparison report.

A.8 Outstanding at issue

The MYOPOOL panel BED paths in `conf/clinical23.config` still refer to the previous installation and must be corrected and re-qualified before the first MYOPOOL run on this server. Twist panel runs are unaffected.